

## Article

# Enhancing Multi-Factor Authentication for Mobile Devices Through Cryptographic Zero-Knowledge Protocols

Thomas Segkoulis<sup>1</sup> and Konstantinos Limniotis<sup>1,2,\*</sup> 

<sup>1</sup> School of Pure and Applied Sciences, Open University of Cyprus, Latsia, Nicosia 2220, Cyprus; thomas.segkoulis@st.ouc.ac.cy

<sup>2</sup> Hellenic Data Protection Authority, Kifisias 1-3, 11523 Athens, Greece

\* Correspondence: konstantinos.limniotis@ouc.ac.cy or klimniotis@dpa.gr

**Abstract:** During the last few years, smart mobile devices have constituted an indispensable part of our lives, being a main element for many daily activities. However, it is well known that several security and privacy concerns still occur, especially taking into account their role as an authentication factor for many users' applications. This paper focuses on multi-factor authentication methods based on mobile devices, proposing a new user authentication scheme based on cryptographic zero-knowledge protocols. This new approach aims to enhance, with minimal effort and cost, any existing authentication method by offering an additional authentication factor based on a unique device identifier through an intuitive and adaptable solution that can be seamlessly integrated into any mobile system, thus providing an additional authentication layer. The ultimate goal is to bridge the gap between ease of use and strengthening security without disrupting the existing infrastructure. A security analysis of the new scheme is presented, whereas an implementation illustrates its effectiveness. It is also shown that this approach is in line with relevant legal data protection and privacy requirements.

**Keywords:** Feige–Fiat–Shamir protocol; mobile device; multi-factor authentication; zero-knowledge proof



Academic Editor: Aryya Gangopadhyay

Received: 27 March 2025

Revised: 27 April 2025

Accepted: 29 April 2025

Published: 30 April 2025

**Citation:** Segkoulis, T.; Limniotis, K. Enhancing Multi-Factor Authentication for Mobile Devices Through Cryptographic Zero-Knowledge Protocols. *Electronics* **2025**, *14*, 1846. <https://doi.org/10.3390/electronics14091846>

**Copyright:** © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

## 1. Introduction

The security of a system is like a chain—i.e., it is as secure as its weakest link [1]. Usually, the weakest part of a system is the human factor, which yields weaknesses that could result in, e.g., social engineering attacks [2]. In this framework, authentication methods have a crucial role in the overall security chain, since compromising authentication yields illegitimate access to the system's data and other resources, which in turn puts confidentiality, integrity, and availability at stake. Despite technological advances in recent years, the provision of secure authentication methods to users so as to address risks stemming from the human factor is still a challenging issue. To this end, having as a starting point the universal adoption of smart mobile devices, such devices have started being a preferable medium for supporting authentication procedures in the so-called multi-factor authentication settings [3].

More precisely, multi-factor authentication is the combination of multiple one-factor authentication types. There are several categories of authentication mechanisms based on, e.g., 'something you know', 'something you have', and 'something you are'. Authentication types based on 'something you know' (e.g., authentication based on a password) are the most frequently used but also the most vulnerable due to the dependence on the human factor. They are very popular because they are easy to implement at low cost, but also

because users are very familiar with them, thus improving the overall user experience. However, it is typically not adequate to rely only on a user's secret, and thus, other authentication factors need to be simultaneously in place—e.g., having also a one-time-password (OTP) being sent to the user's mobile device through an SMS service, thus yielding multi-factor authentication settings, typically relying on a user's device [4]. Many well-known organizations dedicated to cybersecurity, as well as established standards, impose the use of multi-factor authentication techniques in order to reduce potential threats taking advantage of known vulnerabilities (see, e.g., [5]).

Despite the advantages of multi-factor authentication settings, simply combining multiple factors does not solve the problem if each of them has a weakness, as Bruce Schneier states [6]—and this is a common approach, even by financial institutions. Moreover, two factors may not always be adequate: for example, simply relying on the user's (even strong) password for a website, together with an OTP sent to the user's personal registered mobile device, cannot efficiently address the so-called SIM swapping attacks [7]. Furthermore, a systematic investigation [8] of the security of many different implementations of multi-factor authentication schemes for mobile devices in recent years revealed many vulnerabilities and failures. Therefore, it is evident that improving the overall design of the multi-authentication protocols for mobile devices is of utmost importance.

### *1.1. Research Objectives*

The main objective of this research is to propose an appealing solution to enhance existing multi-factor authentication schemes based on mobile devices without the need to change any existing infrastructure. To achieve this, we use cryptographic interactive zero-knowledge protocols whose attributes fit well with the desired security features. More precisely, we suggest introducing the well-known Fiat–Shamir identification protocol (and, more precisely, the Feige–Fiat–Shamir variant) as an additional authentication layer so as to achieve the following: A user who is to be authenticated also needs to possess a specific device (which provides stronger safeguards than the possession of a specific registered phone number). To this end, the notion of cryptographic zero-knowledge protocols ensures that the unique device's identifier (i.e., the Android ID for Android devices or the 'identifierForVendor' (UUID) for iOS devices), which constitutes an authentication factor in our proposed method, is not being transmitted at all during the authentication procedure and, thus, it is neither vulnerable to eavesdropping nor introduces privacy concerns. By these means, we add an extra authentication layer (factor) that is independent of the user's actions or choices and thus, it does not affect the user's experience. A security analysis of this approach is also presented.

### *1.2. Research Questions*

More precisely, this paper focuses on the following research questions:

- Q1. To what extent can interactive zero-knowledge cryptographic protocols be incorporated into existing multi-factor authentication (MFA) frameworks to strengthen their security?
- Q2. What are the advantages of such an approach, and what are the limitations?
- Q3. Can such a solution be practical for realistic scenarios?

### *1.3. Structure of the Paper*

The structure of the paper is as follows. First, the necessary background information is provided in Section 2, describing the various authentication methods with their respective drawbacks, as well as the notion of the cryptographic zero-knowledge protocols. Section 3 presents prior work on this subject (i.e., using zero-knowledge protocols for authentication

purposes), while Section 4 presents in detail the new approach that is proposed in this work. Section 5 illustrates the practical implementation of the proposed approach. An overall discussion and security evaluation are given in Section 6, based on well-determined adversarial models. A discussion about how this approach is compliant with relevant data protection legal requirements is given in Section 7, whereas a comparison with other state-of-the-art zero-knowledge protocols in terms of performance is given in Section 8. Finally, concluding remarks are given in Section 9.

## 2. Background

### 2.1. Authentication Methods

User authentication is the ‘gatekeeper’ process, aiming to verify that a user is indeed the one that they claim to be. Therefore, the security of the authentication process is a vital step towards ensuring confidentiality, integrity, and availability of data.

Authentication mechanisms are typically classified as follows, according to what the user is required to prove:

- Something you know (e.g., passwords or PINs).
- Something you have (e.g., smart cards or SMS-OTP).
- Something you are (e.g., facial recognition or fingerprints).
- Something you produce (e.g., voice, signature or other behavioral patterns).
- Somewhere you are (e.g., locality history or current locality).

More precisely, the authentication mechanism based on ‘something you know’ is one of the most commonly used and the most vulnerable (see, e.g., [9]) due to the great dependence on human factors since the user is the one that chooses and manages the secret information. According to a recent report [10], a large percentage of cyberattacks (greater than 80% of cases) are based on mismanagement or misuse of credentials. The three most common categories of attacks related to stealing or finding user passwords are password cracking attacks, brute force attacks, and dictionary attacks, which are very common, as illustrated in, e.g., [11–13]. Another common attack to retrieve private users’ credentials is the so-called phishing attack, belonging to the broader class of social engineering attacks [14]. A more specialized variation of such attacks is spear phishing [15]. With spear phishing, the payload being sent to the potential victims is tailored to them, specially crafted, containing personal information extracted in a previous reconnaissance step.

Therefore, it is essential to have authentication mechanisms that rely simultaneously on multiple factors; typically, the multiple factors are of different categories, thus making the overall authentication procedure more secure. To this end, the user’s mobile device can support such implementations (for example, apart from a combination of user name and password, the user may also be required to submit an OTP that they receive on their device). However, typical authentication methods relying on ‘something you know’ factors such as one-time passwords (OTPs) and even time-based one-time passwords (TOTPs) have their own weaknesses (see, e.g., [4,16]).

Other authentication mechanisms relying on ‘something you are’, ‘something you produce’, or ‘somewhere you are’ introduce, in principle, privacy concerns, since the authentication server may collect a significant amount of personal data that could possibly be used for other purposes, apart from authentication—and without the user being aware of this. Furthermore, also with respect to privacy and data protection concerns, it should be pointed out that biometric data being used for uniquely identifying a user constitutes a special category of data according to the European General Data Protection Regulation (GDPR) [17], and thus, there exist very strict requirements to ensure the lawful process of these data.

## 2.2. Cryptographic Zero-Knowledge Protocols

Zero-knowledge protocols constitute a family of cryptographic protocols that give the ability to an entity (the prover) to prove that a statement is valid to another entity (verifier) via an interactive communication and without revealing the secrets that are related to the statement. These protocols are based on the old concept of interactive proofs in combination with commitment schemes.

The zero-knowledge proof (ZKP) constitutes the main mechanism for such protocols, establishing the foundation for the interactive scheme. One of the first theoretical frameworks was introduced by MIT researchers Shafi Goldwasser, Silvio Micali, and Charles Rackoff [18] in the 1980s. They established the theoretical background of the interactive zero-knowledge proof and the computational complexity theory supporting it. The introduced zero-knowledge interactive proof scheme was probabilistic in nature as an attempt to minimize the amount of shared knowledge, thus the interactive requirement.

The concept of a ZKP is associated with the following characteristics:

- **Completeness:** the verifier should be able to verify that the statement is valid (even after several interactions) with very high (i.e., overwhelming) probability.
- **Soundness:** the statement should be proved to be correct and thus valid in polynomial time.
- **Zero-Knowledge:** The only information shared with the verifier should be whether the statement is valid or not and nothing more. Even in the extreme and unexpected case where the verifier is manipulating the protocol and returning valid answers without knowing the secrets of the problem, no further knowledge should be extracted.

In principle, these types of cryptographic protocols are attractive due to their nice properties in terms of privacy and security (they constitute a privacy-enhancing technology; see, e.g., [19]).

There are multiple variations of zero-knowledge protocols, each one associated with a different NP-complete or high-complexity problem; indeed, as also stated, the main purpose of a zero-knowledge protocol is to prove that an entity possesses a secret without revealing it, and the proof is derived from the secret, which remains hidden behind a hard-to-solve non-polynomial or NP-complete problem. Fiat and Shamir [20] introduced one of the earliest authentication schemes between a ‘prover’ entity (claimant) and a ‘verifier’ entity, while a well-known variant based on low-cost ZKP is given by Feige, Fiat, and Shamir in [21] (and this is the scheme that has been used, as a case study, in our paper, as shown next).

## 3. Previous Work on Using Zero-Knowledge Proofs for Multi-Factor Authentication

Zero-knowledge proofs (ZKPs) have been extensively studied and deployed in various real-world applications over the past years. Among the different types of ZKPs, non-interactive zero-knowledge proofs (NIZKPs) have gained significant traction due to their efficiency, scalability, and ability to establish secure proof systems without repeated communication between the prover and verifier.

This section summarizes relevant prior work in this area.

A relevant approach utilizing zero-knowledge proofs in multi-factor authentication schemes can be found in [22]; in this paper, the authors present a secure remote multi-factor authentication scheme based on chaotic maps, tailored for the crowdsourcing IoT environment, demonstrating the viability of zero-knowledge protocols to enhance security in distributed systems.

An alternative implementation of ZKPs in authentication for IoT environments is presented in [23], in which an IoT security system that employs a modified zero-knowledge proof algorithm to enhance authentication processes is presented, demonstrating the adapt-

ability of zero-knowledge techniques in securing IoT networks. For a recent survey on ZKP-based authentication in IoT environments, we refer to [24].

Hegde and Manvi in [25] propose the MFZKAP scheme, a multi-factor ZKP authentication protocol designed for secure service access in vehicular cloud computing environments, illustrating the value of zero-knowledge protocols in providing robust, privacy-preserving authentication without revealing sensitive user information.

ZKPs have also played a significant role in enhancing privacy and scalability in blockchain systems, being used especially to ensure that transactions can be verified without revealing any sensitive information. Non-interactive ZKPs, particularly zk-SNARKs (Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge) [26], have been widely adopted in blockchain platforms like Zcash [27] to achieve private and anonymous transactions. Other advancements include zk-STARKs (Scalable Transparent Arguments of Knowledge) [28], which eliminate the need for trusted setup and improve transparency and scalability, making them attractive for broader blockchain use cases.

Very recently, the European Commission issued a call for tenders for the provision of efficient, privacy-preserving age verification mechanisms, based on the necessity to have age assurance for users of digital services according to the relevant legal provisions [29]. To this end, it is interesting enough to point out that this call makes an explicit reference to zero-knowledge proofs.

In all the above cases, the ZKP is being used mainly for authenticating nodes on the networks without revealing either their identity (so as to have, for example, anonymous communication by avoiding transmitting identities) or other authentication factors—such as, e.g., passwords—entered by users (so as to avoid data leakage of such crucial information). In other words, none of these works rely on the secrecy of a device's inherent unique identifier so as to allow authentication of a device itself.

## 4. The Proposed Authentication Scheme

In this section, we present the new authentication scheme based on a zero-knowledge protocol—which, in our case, is the Feige–Fiat–Shamir protocol [21]. The main idea behind the scheme is the additional use of the device's secrets during authentication so that the user proves that the device being used is a legitimate one. Hence, by these means, we introduce an additional authentication factor to existing multi-factor authentication schemes, which relies on the existence of the legitimate user's device without sacrificing the user's privacy.

### 4.1. The New Authentication Scheme in a Nutshell

Before presenting the new scheme in detail, we shall present it in a nutshell:

1. When the user registers for a service, in order to get relevant credentials (e.g., login name and password) and probably provide her phone number as a second factor for authentication (e.g., for an OTP authentication mechanism), she also registers her mobile device itself:
  - The registration of the device does not necessitate the transmission of any of the device's secret information (e.g., the device's ID, etc.); these remain secret.
  - A user is able to register more than one device.
2. Subsequently, whenever the user needs to authenticate herself to this service provider, she performs the same steps that typically the service provider requires (e.g., she provides the password and the OTP, if these are needed by the service provider to perform authentication of the user). However, at the same time, the service also verifies whether the device used belongs to this user or not—i.e., that it has been registered by this user; if this is not the case, then the user is not authenticated.

- This device authentication becomes automatic; no specific user action is required.
- No secret device information is being transmitted to the service provider during this process (due to the zero-knowledge protocol).

To this end, the device's registration and authentication are performed through a ZK protocol, where the corresponding secret is contingent on a unique device's identifier.

Moreover, the proposed scheme has the following properties:

- P1. It can be implemented on any platform without restrictions.
  - In our testing environment, as described next, we are based, simply as a case study, on iOS; however, the underlying platform could also be Android.
- P2. Any ZK protocol can be used, i.e., the underlying logic of the scheme is independent from the choice of the ZK protocol.
  - In our testing environment, as described next, we are based on the Feige–Fiat–Shamir protocol, relying on the hardness of the quadratic residuosity problem in conjunction with the factorization problem; however, our proposed approach can be adapted so as to be based also on other ZK protocols.

#### 4.2. Main Assumptions

Towards developing the new authentication scheme, the main assumptions that we have are the following:

- A1. The service provider is securely authenticated—e.g., through a digital certificate in a PKI setting—in order to avoid Man-in-the-Middle attacks (of course, this is a requirement also present in current authentication schemes).
- A2. The zero-knowledge proof used is cryptographically strong.

#### 4.3. Design Requirements

Towards designing the new scheme, the following requirements have been set:

1. Interoperability with existing authentication schemes: the zero-knowledge protocol should lie at a level under the existing authentication scheme, which in turn should be completely agnostic of its presence.
2. Interoperability with various vendors: the proposed scheme should be compatible with all main device vendors and mobile operating systems (i.e., Android and iOS).
3. Multi-user device: consideration that a single device might be used by multiple users.
4. Pluggable zero-knowledge protocol: the overall architecture of the proposed scheme should be abstract enough, allowing any zero-knowledge protocol to be plugged in and used in an easy way.
5. Efficiency: the proposed scheme should provide an efficient way to establish a communication channel for the interactive zero-knowledge protocol.
6. Security and privacy: user's personal data, including device identifiers, should be protected.

To address the aforementioned requirements, the proposed scheme (a) encapsulates the authentication payload that is created by the existing authentication method (whatever this method is) adopted by the service provider, along with any additional request header values, and (b) executes the zero-knowledge protocol client verification process as an additional step. Hence, the overall architecture follows the well-known open-closed principle [30] from software engineering, where a software module should be open for extension but closed for modification. Moreover, the choice of a zero-knowledge protocol is to ensure that no personal data (user or device information) ever leaves the device and is transmitted to the public network (this is an inherent feature of the zero-knowledge protocols). Nevertheless, specific device information, uniquely associated with a specific



user, is being obfuscated, while other user secrets are being saved locally in secure storage (e.g., keychain or secure enclave).

Finally, it should be pointed out that the scheme's main function is the enhancement of the authentication, leaving the heavy workload, i.e., the public key infrastructure (PKI) and digital certificate managing, to the existing main authentication scheme.

#### 4.4. Nature of Secret Being Used

In our proposed scheme, the main secret that is being used is the unique device identifier (i.e., a possession secret). Such an identifier, based on the application vendor and the device, could be used to verify the eligibility of the device in conjunction with other multiple authentication factors. However, since a device might be used by multiple potential users, such a device secret should be associated/mixed with a user's identifier (as it will be discussed next).

#### 4.5. System Entities

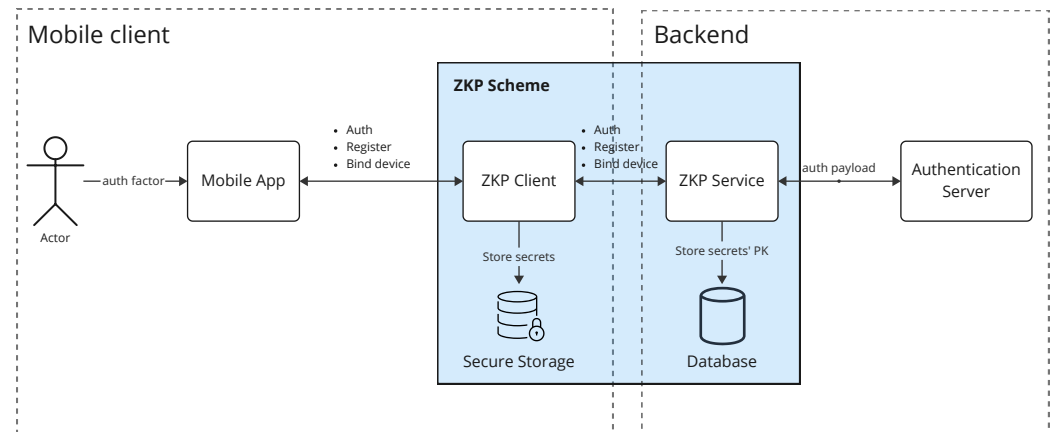
The proposed scheme consists of two main domains of entities, one covering the client side at the user end and a second one concerning the server side, which performs as the remote authenticator.

More precisely, the entities are as follows:

- User: this is the end user attempting to authenticate herself through a mobile device and get authorization to access the system's resources.
- Mobile Device: This is the mobile device acting as the prime multi-factor authentication medium. The proposed scheme aims to take advantage, in a secure and private way, of any device identifier private information or values related to the registration procedure that could potentially be used in the verification step later on.
- Mobile Application: A mobile application in the context of the current proposed scheme could have two roles. It could either be a standalone multi-factor authentication executor with the sole responsibility of authenticating a user, like the typical OTP applications, or a full-scale client-side application providing services and features after a user has been granted the appropriate access control rights; in the latter case, the authentication step is just one of the features that the app provides.
- Server: This is the server-side application performing user authentication via web services. Similar to the mobile application, it could either be a standalone web service having the single responsibility of authentication and authorization or a generic application providing services while having authentication embedded into it. The nature of the authentication method is to be independent of the proposed scheme since the proposed scheme is a plugin in a lower level of the stack.

#### 4.6. System's Architecture

The proposed scheme consists of two main modules: (a) the mobile client side that initiates the authentication process and (b) the server side that responds to client requests and operates as the authenticator. Those modules act upon an upper-layer authentication request of arbitrary nature and technology. The authentication request encapsulates the original authentication payload (i.e., the payload related to the typical user's credentials for accessing the said service) and initiates a zero-knowledge protocol. This protocol is used to prove the validity of the device that triggered the initial request (which should be associated with a secure registration process that has already taken place with this specific user). The device is valid if it possesses secrets shared during user registration that have already taken place earlier. Figure 1 illustrates an abstract overview of the architecture.



**Figure 1.** The system's architecture.

More specifically, in our scheme, there is a library at the client side (ZKP client library) that provides four main functionalities: (i) new user registration, (ii) existing user authentication, (iii) new user device binding, and (iv) adding new user secrets. This library is completely agnostic of the nature of the authentication payload received from the upper layer, and just encapsulates it into its own requests to the server (similarly to the TCP/IP layer communication). The reverse communication to the main application is established via callbacks triggered after an operation has finished, either with success or failure, providing detailed information.

Similarly, on the server side, the scheme provides a server-side plugin that acts on the client's requests, performing the role of the prover in the zero-knowledge context and passing the authentication payload to the authorization server (note that both the authentication and authorization servers rely on the server side). The payload received remains intact in this layer since it is targeting the upper authorization server layer and should be properly encoded and encrypted. The plugin provides two main API interfaces to communicate with the client depending on the operation requested and the zero-knowledge protocol algorithm selected. These are as follows:

- HTTP requests, which are being used when a simple request and response suffice to execute an operation. Otherwise, if multiple requests and responses are required, like in the case of an interactive zero-knowledge challenge, then a WebSocket connection is established (see below) to facilitate the exchange of data (for non-interactive zero-knowledge protocols, a simple HTTP request should also be sufficient).
- WebSocket connections that are tailored to specific use cases. The execution of the ZKP is one of them, since it is required to have multi-step back-and-forth communication. WebSocket utilizes HTTP requests to establish and maintain a persistent communication channel, minimizing the overhead of repeatedly opening new connections. This ensures synchronous and secure communication, in terms that both client and server are confident in their ongoing interaction throughout the challenge.

#### 4.7. Main Operations

In this subsection, we present the main operations that constitute the basis for the new enhanced authentication scheme based on ZKP.

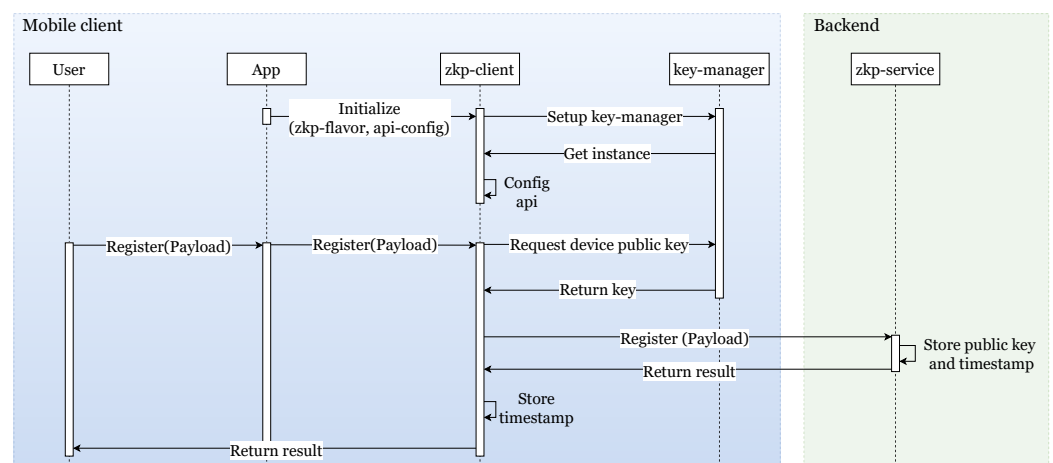
##### 4.7.1. Registration

This is the first operation (Figure 2) in which a new user is registered to the desired services. During a typical registration process, the user provides personal information and 'creates' unique credentials in a secure way, according to the typical procedure that



the service provider adopts for registering users. These credentials will be used later to uniquely identify, authenticate, and eventually authorize the user to perform certain actions. In the proposed scheme, existing registration procedures are not affected; however, we add a new layer that encapsulates the aforementioned credentials as payload into a new request that is being sent along with zero-knowledge public keys. Those public keys will be used in the subsequent operations to identify and verify the eligibility of the device per user, asserting that this device is the one that has already been used during the registration process by the registered user.

Before proceeding by describing the registration process in detail, it should be explicitly clarified that, in our scheme, the user is being registered to the service by exactly the same means as it currently happens per service provider. For example, if a bank requires that a person should be identified physically, through, e.g., presenting an identity card at a bank store, in order to get a login name and an initial key number so as to subsequently access the web banking and choose her own password (thus completing the registration process), then this procedure is not affected at all. Our scheme simply adds a new authentication factor uniquely associated with the device that the user has during her registration (i.e., in our example for the web e-banking, with the device that she uses to complete the registration). Therefore, in fact, we do not alter the user registration process, but we ‘enrich’ it with a device registration that occurs simultaneously with the user’s registration.



**Figure 2.** Registering user’s device with the proposed ZKP scheme.

From an implementation perspective, the registration phases are as follows:

1. The app initializes ZKP client: The host app containing the ZKP client library initializes it by selecting the zero-knowledge protocol to be used. In addition, this is the point to configure the remote ZKP service, setting the server’s host base URLs for the HTTP services and the WebSocket connections.
2. The user initiates registration: The user triggers a new registration by entering their personal information and unique credentials into the app. The nature of these credentials is irrelevant to our scheme. This actually coincides with a typical registration process.
3. The app sends the registration payload: The user’s application encrypts and encodes the registration payload along with the user’s identifier and sends it to the ZKP client library. The actual content of this payload is agnostic to the ZKP client library; it is the same payload that would be sent to the application’s remote service to complete registration in a typical scheme.
4. Generate device ZKP public key: The ZKP client instructs the key management component to generate and return the ZKP public key, taking into account the client’s

secret. This secret is derived by the concatenation of a user's unique identifier with a device's unique identifier. The choice of a unique device identifier ensures that the device is uniquely identified.

5. Send registration data to server: Assemble the app's registration payload with the ZKP device's public key and send it to the respective remote ZKP service via an HTTP-POST request. The parameters in this POST request also include the user's identifier and the selected zero-knowledge protocol in order for the server to be able to execute the verification challenge later on, upon the client's request.
6. The server processes the registration payload: The server receives the request to register a new user, and if the registration info is correct, then the new user is registered, and it further proceeds with storing the ZKP public key mapped to the specific device (and, effectively, to this specific user) into ZKP service's local database. The timestamp of the request execution is also stored on the server side. The app's encapsulated registration payload is transmitted and processed by the client's server as if we did not have this new layer. The ZKP service layer is a low-level layer on the backend side with the sole responsibility of processing requests and executing the zero-knowledge protocol scheme; hence, the pre-existing registration logic remains unchanged.
7. The client receives a callback: After the remote service finishes registering the user and storing the respective ZKP keys, the client receives a callback with an indication of success or failure in case of an error. As a post-request action, the client stores the device's ZKP public key into the secure storage for quick access, as well as the registration timestamp.

The above procedure ensures, as it will become evident next that the server will be able to authenticate the user not only according to the, for example, 'traditional' authentication methods—such as, e.g., password and OTP—but also according to the device that the user has; if the user tries to authenticate herself from a different mobile device, then the authentication process will fail. The important property here is that to achieve this, the authentication server does not need to learn any secret device information due to the zero-knowledge protocol.

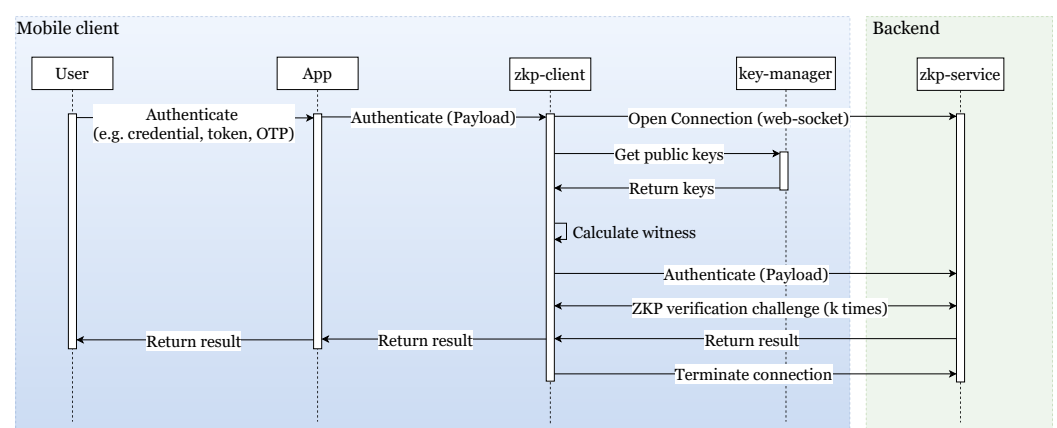
#### 4.7.2. Authentication

This operation (illustrated in Figure 3) is triggered when the user initiates authentication by any authentication factor (e.g., by entering her corresponding credentials). The ZKP client gets the authentication payload generated by the application and after adding the ZKP public keys, it transmits it to the remote ZKP service. The ZKP client handles the payload as a black box and is completely agnostic of its content. This is the main part of the scheme in which the zero-knowledge protocol is used as an auxiliary method of protection against unauthorized actors—e.g., against those who have acquired user credentials through phishing attacks. The execution of the zero-knowledge protocol's verification asserts the device to be an eligible one that has already been registered directly (first time) or indirectly (bound via another registered device) for the specific user.

From an implementation perspective, the corresponding phases for this task are as follows:

1. The user initiates authentication: the user starts a one-factor authentication (e.g., username/password, OTP, token) in the main application.
2. The app sends an authentication payload: the user's application encrypts and encodes the authentication payload along with the user's identifier and sends it to the ZKP client library (the library cannot see the content of the payload, as it is the same payload that would be sent to the application's remote service to complete the authentication process independently from the library).

3. Connect to remote service: the ZKP client opens a WebSocket connection to the remote ZKP service.
4. Fetch ZKP public keys: The ZKP client fetches the ZKP device's public key already shared with the remote service from the key manager to be sent along with the authentication payload. The ZKP service at the server side requires this key in order to identify the device bound to the specific user and execute the verification challenge.
5. Create a witness of the ZK protocol: choose a random and ephemeral number  $r$  and, based on  $r$ , create a witness to be used by the execution of the zero-knowledge protocol (according to the requirements of the Feige–Fiat–Shamir, as described next in Section 5.2.1).
6. Send authentication message: Now, the ZKP client has all the necessary information in order to assemble the app's authentication payload; this will consist of the ZKP device's public key, the user's identifier, and the witness. Then, this authentication message is sent to the respective remote ZKP service at the server as the first message within the WebSocket connection. The parameters also include the selected zero-knowledge protocol in order for the server to be able to execute the verification challenge later on, upon the client's request.
7. Execute interactive ZKP challenge: At this phase, the verification of the user's device takes place by exchanging back-and-forth messages in the context of the interactive ZKP. More precisely, the remote service sends a challenge to the client, who in turn replies with a response to this challenge, based on the secrets it possesses and the secret random value  $r$ . During every exchange, the ZKP service checks the validity of the client's response and either proceeds to send a new random challenge (if the response is correct) or completes the session (if the response is wrong).
8. Return callback: At the end of the ZKP challenge, the remote service returns a successful callback to the client, provided that all the client's responses were correct; otherwise, it returns an error during the challenge and informs the user of an authentication failure (possibly due to a fraudulent action—i.e., in case of a masquerade attack based on a non-genuine device that fails to prove that it has knowledge of the correct secret value).
9. Connection termination: The WebSocket channel is terminated by the client.



**Figure 3.** Authentication sequence diagram.

#### 4.7.3. New Device Binding

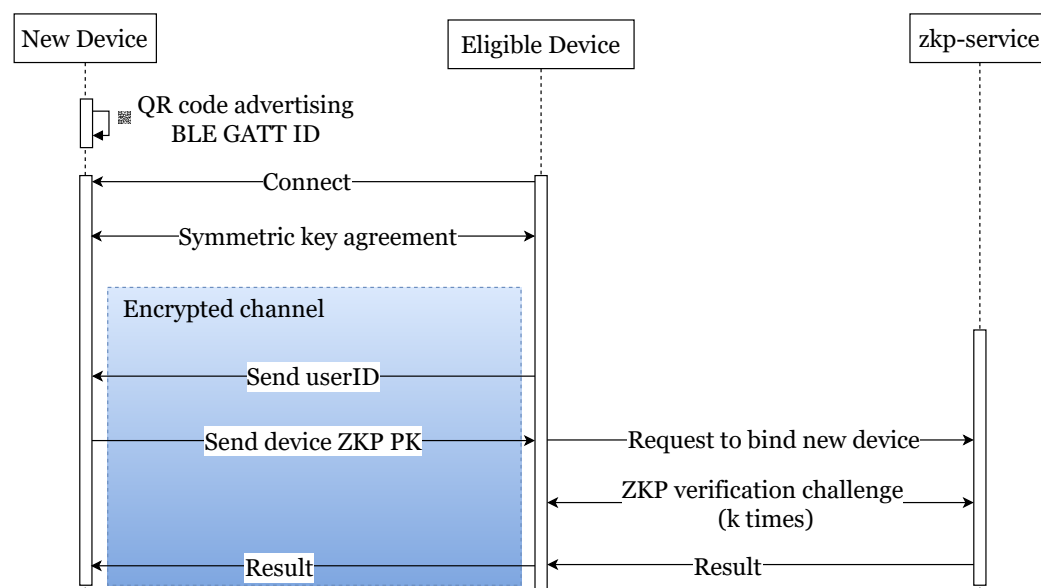
The above registration and authentication procedures aim to ensure that the user will not be authenticated to a service by simply entering her credentials or providing other additional authentication factors such as, e.g., OTP; it is additionally required to be used, during the authentication process, the same device that has been used in the registration

process. However, this may lead to practical problems in accessing services since, typically, the users have more than one device. To alleviate this issue, our scheme allows multiple devices per user that can also be registered as eligible and be able to be verified via the zero-knowledge protocol; we call this procedure new device binding. In order to ensure that the new devices that will be bound are indeed legitimate devices for the said user (i.e., that a binding request is being sent by the legitimate user and not by an adversary), the binding of these devices can only take place via devices that have already been registered and under specific conditions. In particular, the process takes place when the two devices are in close proximity using Bluetooth wireless technology and, more precisely, Bluetooth Low Energy (BLE). The communication specifications are based on the Generic Attribute Profile programming interface (GATT).

The process of binding is based on guidelines and specifications of the BLE technology and the included GATT protocol. The key elements of such a wireless system are the following:

1. Peripheral: a device that publishes services from its local database and advertises them to the central devices.
2. CentralManager: a central device that can detect and connect to peripheral devices and subscribe to their services' characteristics.
3. Service: A group of data emitted by a peripheral device available for central devices after they subscribed for them. The data within a service are semantically related.
4. Characteristic: The single data value, as specified by GATT, is to be transmitted by the peripheral. They are semantically grouped into services.
5. DeviceBindingMessageDTO: Statically typed low-level data object. It is used to formalize the exchanged messages within the characteristic data between peripheral and central devices. It contains a payload as plain binary data and a type (syn, ack, waitingForPK, sendingPK).

Hence, in our framework, the phases for binding new devices are as follows (Figure 4):



**Figure 4.** New device binding sequence diagram.

1. Setup new device: The new device registers as a peripheral, setting up its services and waiting for central devices (an eligible device) to connect. It advertises its services and characteristics via a QR code containing a URL in a deep-link format (i.e., URLs triggering an internal action to the app). In the current scenario, the deep-link URL

contains the advertised service-id, making the central device capable to connect and start the next phase upon scanning the QR code. It is not a prerequisite for the new device to have attempted to authenticate.

2. Symmetric key exchange: This is the next phase after establishing the BLE connection between the new device and the already eligible one (having the role of the “authenticator”). Both parties exchange keys and generate a shared key by performing NIST P-256 elliptic curve Diffie–Hellman (ECDH). The “authenticator” initiates the key exchange by sending its public key and timestamp in a message of “syn” type. The new device responds with an “ack” message including its own public key and the timestamp retrieved by the previous “syn” message. From now on, an encrypted channel is established, and the two parties can use the shared key to securely communicate private information.
3. Send userID: The eligible device or “authenticator” sends a message to the new device requesting its ZKP’s public key based on its unique device ID. The missing userID is transmitted by the currently logged-in user. The new device will be registered as an additional eligible device for the same user.
4. Send device ZKP public key: the new device generates and sends back the ZKP public key based on its unique device identifier and the userID it has just received.
5. Request to bind new device: The eligible device opens a WebSocket connection to the remote ZKP service that has the role of the verifier requesting a new device binding. It transmits the new device’s ZKP public key, which is obtained by the new device’s unique identifier similarly to the procedure that exists for the registration.
6. ZKP verification challenge: After the request goes through, in the same channel, the eligible device proves to the ZKP service that it is an eligible one and not a malicious actor by executing the ZKP verification challenge. The verification is the same as the one executed during authentication.
7. Completion: After the ZKP service accepts the new device’s ZKP public keys, it sends a callback to the eligible device informing it accordingly. Then, the eligible device closes the WebSocket connection with the remote service and sends a callback to the new device, marking the successful binding, and the BLE connection is terminated. In case of errors or problems executing the request, the eligible and the new device are informed accordingly via failure callbacks.

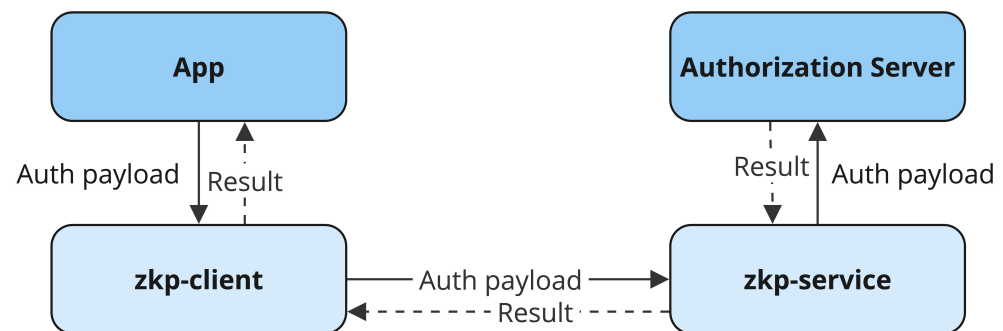
Security in this setup primarily relies on the close physical proximity of the devices, ensuring that only co-present devices can be securely added to the pool of trusted devices, while the legitimate user needs to scan a QR code to allow the binding of the new device. To further enhance this mechanism, the QR payload could possibly include additional identifying metadata such as the device model and name, while also incorporating a short-lived timestamp or nonce to prevent reuse or replay attacks by ensuring the QR code expires after a brief validity window.

#### 4.8. The Integration of the ZKP into Existing Authentication Architecture

One of the main attributes of the proposed solution is its applicability to any pre-existing authentication architecture between client and server by simply adding an extension. Hence, the solution can be easily adopted without introducing significant changes to the existing infrastructure, thus facilitating its development and its future changes. Moreover, there are no restrictions on the choice of the zero-knowledge protocol that can be used; by these means, the new scheme should be versatile enough to support even new future zero-knowledge algorithms.

The above are illustrated in Figure 5. More precisely, Figure 5 depicts the integration of the proposed ZKP into an existing client-server architecture performing authentication.

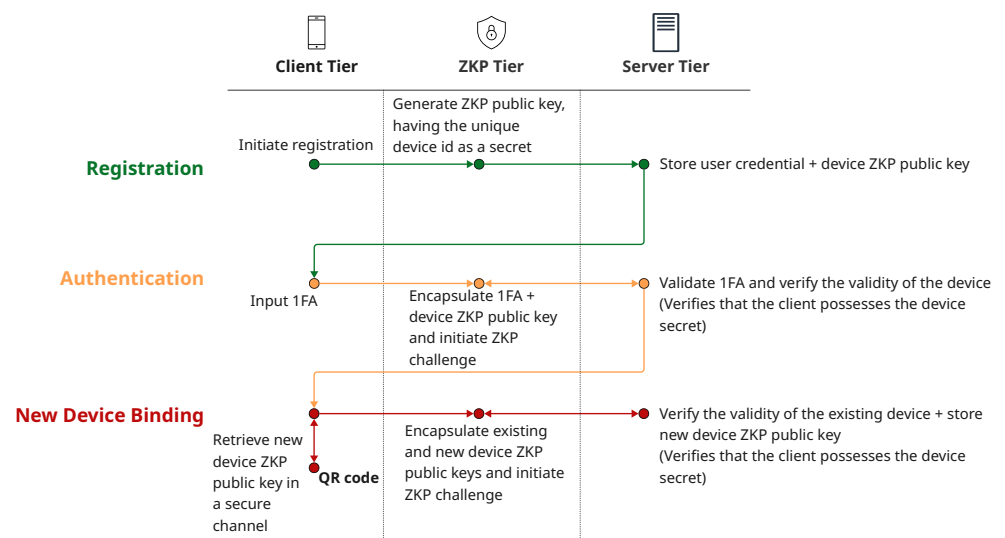
Note that the application's authentication payload does not affect this new layer, which is independent of the upper-layer client-server architecture authentication (i.e., any of the most common technologies, like username/password, passkeys, etc., can be used).



**Figure 5.** Extending authentication with the proposed ZKP scheme.

Note that the authorization server is completely agnostic of the source of the authentication payload—something that makes the solution ‘pluggable’ on the server side as well.

Figure 6 illustrates the aforementioned three main phases (registration, authentication, and new device binding) at a high level of view. The term ZKP tier refers to the whole ZKP functionality, including both the ZKP client and ZKP service in Figure 5.



**Figure 6.** The three main phases (registration, authentication, and new device binding).

## 5. Implementation

We next proceed by implementing the aforementioned scheme into a test environment, following the architecture detailed in Section 4 as a blueprint. As stated above, the functionality of our scheme is independent of the ZKP that will be used, as well as from the underlying platform. In our case study, the selected zero-knowledge protocol used is one of the earliest, i.e., the so-called interactive Feige–Fiat–Shamir introduced in [21], and the provided operations are as previously described, i.e., registration, authentication, and new device binding. The client side (prover) is implemented as an iOS library, and the backend side (verifier) as a Django framework. Of course, the proposed technique can be implemented in any mobile platform for arbitrary zero-knowledge protocol.



### 5.1. Notation

With respect to the Feige–Fiat–Shamir zero-knowledge protocol, we make use of the following notation:

- $N$ : the product of two large prime numbers, constituting part of the client's (i.e., device's) public key.
- $p, q$ : Two large prime numbers used as factors of  $N$ , constituting part of the client's private keys. As illustrated in [21],  $p, q$  should satisfy the following:  $p \equiv 3 \pmod{4}$ ,  $q \equiv 3 \pmod{4}$ .
- $s_d$ : the device's private key (i.e., the secret of the prover) that is derived by the unique device identifier (as specifically described next in Section 5.2).
- $v_d$ : the quadratic residue (mod  $N$ ) of the  $s_d$  (part of the device's public key).
- $r$ : a random number that will be used to derive the witness in the protocol.
- $k$ : the number of repetitions of the challenge–response protocol, being a security parameter.

The prerequisites are the following:

- The  $s_d$  and  $N$  shall be co-prime (i.e., their greatest common divisor is equal to 1)—which simply means that  $s_d \neq p$  and  $s_d \neq q$ .
- The client (prover) has already shared the public keys  $v_d$  and  $N$  with the server (verifier) during registration. These constitute the device's ZKP public key.
- The client sends an authentication request, including the user's unique identifier and the ZKP public key.

We should recall at this point that the security of the Feige–Fiat–Shamir ZK protocol is contingent on the difficulty of finding square roots of quadratic residues (squares mod  $N$ ), as well as on the difficulty of factoring  $N$ , for large values of  $N$  [21].

### 5.2. The Mobile and Backend Clients

The mobile client of the scheme (ZKP Client) has been implemented as an iOS library that integrates seamlessly into a host project, providing an additional layer of protection during authentication. It receives the encrypted authentication payload and executes the Feige–Fiat–Shamir zero-knowledge protocol, verifying the eligibility of the device.

The zero-knowledge public key is calculated based on the following formula:

$$v_d = s_d^2 \pmod{N}, N = p \times q$$

The device secret  $s_d$  is calculated by concatenating the unique device's identifier with the user identifier and adding required padding at the end in order to ensure a 2048-bit size. The device vendor identifier ('identifierForVendor' in iOS) is a unique identifier that remains the same for an iOS application (or another application from the same vendor) in a device and could be reset after re-installation (similarly, in Android systems, such an identifier could be the so-called Android ID). It constitutes a good candidate to be part of a secret used to verify the origin of an authentication request since this identifier is not known to anyone. In relation to authentication categories, it could fall under the 'Possession Factor'.

A backend service (ZKP Service) is implemented in Django (v. 5.0.1), which is a broadly used high-level Python (v. 3.12.1) web framework. The implementation provides one HTTP-POST request for the registration operation and two WebSocket connections for the rest of the operations (authentication, new device binding).

### 5.2.1. Interactive Zero-Knowledge Authentication Scheme

To initiate a challenge–response message exchange, the client generates a random and ephemeral number  $r$  for the current session of 2048-bit size. The client sends to the server a number  $x$  (witness) derived from  $r$ , calculated based on the formula:

$$x = r^2 \pmod{N}$$

Then, the server sends to the client a ‘challenge’, which is a random number  $e \in \{0, 1\}$ . In response to this, the client calculates

$$y = r \times s_d^e \pmod{N}$$

and sends  $y$  back to the server as a response. The server retrieves the challenge response  $y$  and asserts:

$$y^2 \pmod{N} \equiv x v_d^e \pmod{N}$$

The server sends the assertion response to the client. If the assertion does not hold, then the authentication fails (i.e., the device has not proved its proper identity) and the whole procedure stops; otherwise, this process is repeated  $k$  times (where  $k$  is a security parameter).

### 5.3. Performance Evaluation

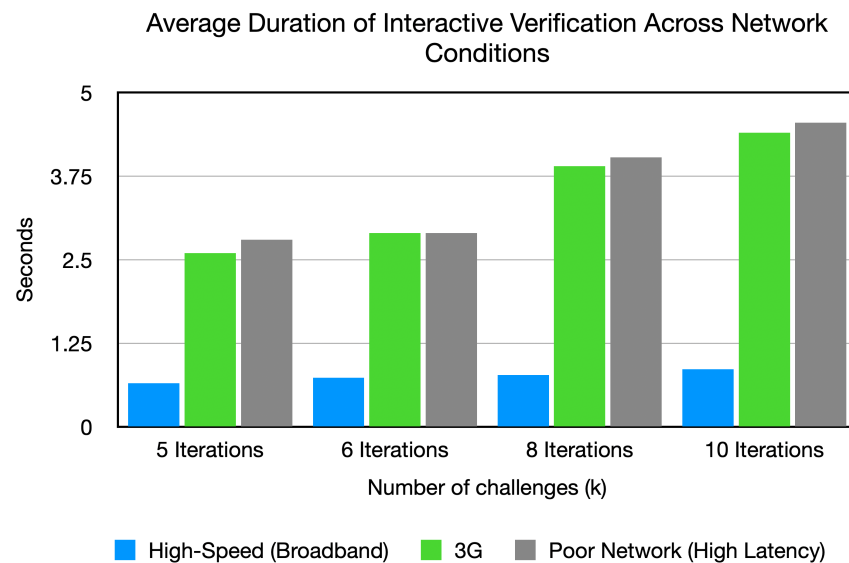
To evaluate the performance of the proposed zero-knowledge protocol, a series of experiments were conducted to measure the average duration of the protocol’s interactive verification process. This process entails multiple rounds of communication between a client and a server. The computational setup consisted of an iPhone application serving as the prover and a local Django server acting as the verifier. The relevant codes we developed are available in [31] (a demo, server-side application, acting as the verifier in a Feige–Fiat–Shamir interactive zero-knowledge verification scheme), ref. [32] (an iOS package implementing the prover side), and [33] (a demo, client-side application, acting as the prover in a Feige–Fiat–Shamir interactive zero-knowledge verification scheme).

The experiments were designed to capture the protocol’s behavior under different network conditions. Initially, tests were conducted under a “happy path” scenario, assuming favorable network parameters that reflect high-speed broadband connections typically found in Europe. To broaden the evaluation, two additional network conditions were emulated using Apple’s Network Link Conditioner: a standard 3G connection and a degraded network with high latency and packet loss, representing a worst-case scenario. The specific parameters used for each of the three network scenarios—high-speed, 3G, and poor network—are summarized in Figure 7. These configurations were applied to the test device using Apple’s Network Link Conditioner to emulate realistic and challenging connectivity conditions.

| Network Link Conditioner Configuration |                        |     |                             |
|----------------------------------------|------------------------|-----|-----------------------------|
|                                        | High-Speed (Broadband) | 3G  | Poor Network (High Latency) |
| In Bandwidth (Kbps)                    | 30,000                 | 780 | 2000                        |
| In Packet Loss (%)                     | 0                      | 0   | 2                           |
| In Delay (ms)                          | 20                     | 100 | 100                         |
| Out Bandwidth (Kbps)                   | 15,000                 | 330 | 256                         |
| Out Packet Loss (%)                    | 0                      | 0   | 2                           |
| Out Delay (ms)                         | 20                     | 100 | 100                         |
| DNS Delay (ms)                         | 0                      | 0   | 0                           |

**Figure 7.** Network parameters used to emulate high-speed (broadband), 3G, and poor network (high latency) conditions via Apple’s Network Link Conditioner.

For each configuration point (i.e., varying the number of interactive challenges, denoted by the security parameter  $k$ ), ten tests were performed, and their average duration was calculated. These average results, depicted in Figure 8, demonstrate the protocol's scalability and the efficacy of WebSocket connections under realistic network conditions.



**Figure 8.** Time required for executing the ZK protocol for various numbers of repetitions.

As expected, the verification time increases with the number of iterations. Under high-speed broadband conditions, the protocol performs efficiently, with average durations remaining well below one second even at 10 rounds. On 3G and degraded networks, performance degrades gracefully, always being within 3–4.5 s. This behavior indicates that fixed network latency is the dominant factor in overall verification time, rather than computational complexity. The underlying cryptographic operations are executed efficiently in polynomial time, suggesting that the protocol's scalability is primarily constrained by communication overhead. These findings confirm the protocol's usability in real-world mobile environments and motivate further optimizations—such as round-trip reduction or batching—to enhance responsiveness under adverse network conditions.

## 6. Security Analysis—Discussion

From the above analysis, it becomes evident that the security and privacy features of the proposed scheme are inherited from the nice cryptographic properties of the zero knowledge proofs. In fact, our scheme is able to enhance any existing authentication mechanism based on mobile devices by adding an additional authentication factor that relies on the authentication of the device itself (without affecting the existing authentication factors). We subsequently present in a more systematic way how security and privacy goals are being achieved.

It should be stressed that since the proposed method adds an additional, device-based authentication factor to any existing authentication scheme, security aspects stemming from any pre-existing infrastructure are still in place. For example, if the registration process that has been currently adopted by a service provider is considered secure, then this will apparently remain secure after the modifications implied by our scheme. The main advantage of our scheme is that, due to this additional authentication factor, an attacker will need more than the legitimate user's credentials—including any credentials that are provided/generated through the user's device, such as OTPs—to masquerade as

this user; more precisely, the attacker will additionally need to have in her hands the user's specific device.

We shall next describe specific security features of the new scheme, starting from properties of the ZKP adopted in our use case (i.e., the Feige–Fiat–Shamir protocol) and continuing with a discussion of how well-known threats can be thwarted due to the properties of our scheme. A discussion on the provable security of the ZKP follows at the end of this section.

### 6.1. Properties of the Zero-Knowledge Protocol

We first explicitly present how the desired properties of a ZKP are being met in our case, which is based on the interactive Feige–Fiat–Shamir protocol. Of course, as also stated earlier, other ZKPs with nice properties could also be an option for our scheme.

#### 6.1.1. Soundness

Let us assume that an attacker aims to masquerade a valid user. Then, first, she needs to know the user's identifier and the ZKP public keys. However, even if the attacker intercepts somehow pieces of information, she will still fail to execute the ZKP correctly; this is due to the fact that she will need to prove that she possesses the legitimate user's device identifier also or the user's private key  $s_d$  derived by this identifier. Therefore, since the attacker does not have the user's device in her hands or does not know the user's unique device identifier or the corresponding private key, any masquerade attack will fail to pass the zero-knowledge protocol.

Of course, an attacker could try to execute the protocol by using her own ZKP public keys (associated with the corresponding attacker's private key). However, this procedure will not have any success since the server will not match this ZKP key in the database, thus illustrating that this key has not been shared during a valid commitment phase (registration or new device binding) from an eligible device; thus, it is not a valid ZKP key. Here, it is important to recall that the typical user's registration procedure remains intact—i.e., if the malicious user Eve wants to register for a service as Alice, using her own device pretending that it is Alice's device, then she still needs to convince the web service from the beginning that she is Alice, bypassing the typical authentication means that the web service has adopted.

As previously discussed, the likelihood of malicious extraction of secrets used by the legitimate user shall be considered as significantly low, as they are securely stored within the device's protected storage or dynamically generated by the operating system at runtime, ensuring vendor-specific uniqueness and non-disclosure. In addition, using a strong hash function—i.e., one-way and collision-resistant—for deriving the user's secret key  $s_d$  from these original secret values ensures that such a valid  $s_d$  cannot be extracted by a malicious actor.

Note also that, for our case study, which relies on the Feige–Fiat–Shamir zero-knowledge protocol, if an attacker could predict the challenge  $e$ , then they could masquerade a legitimate user even without the knowledge of the user's device secret or the secret key  $s_d$ . Indeed, this could be achieved by sending an appropriate witness that will allow her to subsequently provide a response that will be accepted as correct; this is a known issue in the Feige–Fiat–Shamir protocol [21], and that's why the number of repetitions  $k$  is being considered as a security parameter (and it is essential that the witness is being sent before the challenge becomes known). Indeed, the attacker needs to guess correctly all challenges in order to achieve a successful masquerade attack; if the challenge is a fully random value from the set  $\{0, 1\}$ , then the probability that the attacker predicts all

challenges is equal to  $(1/2)^k$ ; this becomes extremely small for values  $k \approx 20$ , while even smaller values may be considered as sufficient.

#### 6.1.2. Completeness

A device storing secrets that were used to generate the public keys is always able to prove its identity with 100% accuracy; this is an inherent property of the Feige–Fiat–Shamir scheme.

#### 6.1.3. Zero Knowledge

Again, due to the inherent property of the Feige–Fiat–Shamir protocol, the server (verifier) is able to verify that a user's (client's) response is correct, despite the fact that it has no knowledge of the secret information that is being used by the user's device in order to generate this response.

### 6.2. Security Features: Confidentiality and Integrity

Private user and device information used as secrets in the Feige–Fiat–Shamir's ZKP are never revealed or even inferred by any means; from a mathematical point of view, deriving such information necessitates, as also stated above, the computation of modular square roots having unknown modular factorization of large prime numbers, which is known to be a difficult problem.

Note that the device's unique identifier is fetched directly by the operating system at runtime and is unique for a specific vendor on a device and never transmitted. Therefore, there is no direct way or other practical means for an attacker to retrieve it.

With respect to other secrets generated during a commitment phase, these are suggested to be stored in secure storage properly encrypted (keychain), ensuring both their confidentiality and integrity.

Confidentiality and integrity of transmitted data are achieved by establishing a secure WebSocket connection over Transport Layer Security (TLS). TLS provides a secure connection between prover and verifier through authenticated encryption. Moreover, during the handshake, apart from exchanging cryptographic keys, the web server is authenticated using trusted digital certificates.

### 6.3. Adversarial Model and Threat Analysis

Next, we proceed by describing more specifically how the proposed scheme manages to address specific, well-known threats.

#### 6.3.1. Adversarial Models

To describe the security properties of a particular protocol, specific adversarial models have been proposed in the literature. One of the most commonly used adversarial models for communication channels is the so-called Dolev–Yao (DY) model [34], which assumes that the entities communicate over a public channel, where the communication is intercepted by the probabilistic polynomial-time adversary and may modify/delete/insert data into this communication; to this end, it is assumed that the adversary may use any information obtained from previous messages, not constrained by other assumptions.

Although the DY model has indeed been the basis to evaluate several mobile device authentication protocols (for a nice survey, see [35]), the special nature of mobile device authentication employs several security threats that are not well covered by this model; for example, the so-called SIM-swapping attacks cannot be addressed efficiently by this model; similarly, as stated in [35], authentication on mobile devices is usually performed in public spaces, potentially under video surveillance, which also poses special security threats. This is the reason that there are several adversarial models for mobile device authentication

methods, hardly comparable to each other since each of them actually focuses on different aspects [35].

In our case, we are based on the DY model in order to illustrate the security of the system, as described next (see Section 6.3.2); thus, the adversary may be any possible malicious third party with the aforementioned capabilities. However, we also illustrate the strength of the proposed approach over several attacks, such as the SIM swapping attacks, that go far beyond the DY adversarial model. Moreover, despite the fact that the genuine authentication server is not considered malicious in our case, our scheme ensures that the server does not learn anything more than the least possible information in order to alleviate privacy concerns; therefore, with respect to the authentication server, we will describe that our scheme is secure under the so-called honest-but-curious (or semi-honest) model—i.e., the attacker (in this case, the server) is a legitimate entity that follows the specified protocol but can read all the transmitting information, trying to learn/infer all possible information from legitimately received messages (see, e.g., [36]).

### 6.3.2. Relevant Threats and How They Are Being Addressed

We next present in more detail how well-known threats are being addressed by the proposed authentication mechanism, according to the aforementioned adversarial model:

- **Man-In-The-Middle attacks:** Secure connection over TLS is an effective solution against Man-In-The-Middle (MITM) attacks, providing several security attributes like digital certificate validation during handshake. As an additional measure, certificate pinning [37] could be deployed, ensuring the authenticity and integrity of the certificates. In any case, the ZKP ensures that no sensitive or private information is disclosed.
- **Social engineering attacks (phishing attacks, shoulder surfing, etc.):** Classical social engineering attacks (e.g., phishing attacks) that extort users' authentication credentials, private information, or keys will have no effect on the proposed enhanced authentication scheme. This kind of attack relies on the vulnerabilities of the human factor (e.g., via social engineering tactics). This is exactly where the proposed scheme comes into place as a powerful mechanism by adding an extra layer of security based on something the user has that is never transmitted or revealed, even to the server.
- **Replay attacks:** Even if a malicious entity somehow intercepts the message exchange during the ZKP verification phase, they will not be able to successfully use this information in the context of a replay attack due to the inherent features of the Feige–Fiat–Shamir protocol; indeed, the challenges being sent by the server are random and unpredictable (the unpredictability of the challenges is an essential feature that shall be ensured), while a valid response to a specific challenge will not be valid for another challenge.
- **SIM swapping attacks:** SIM swapping attacks can be mounted whenever an attacker manages to gain access (i.e., through data breach incidents or through phishing attacks) to large amounts of a user's personal data, including the user's identification information or even the whole user's ID card, phone number, e-mail address, and password. Then the attacker needs only to have in their hands a mobile device with the victim's phone number so as to bypass the two-factor authentication required for, e.g., banking transactions; to achieve this, the attacker mimics the victim by requiring a new SIM card from the victim's telecommunications service provider, claiming that they have lost their device. If the service provider fails to effectively identify this fraudulent request, then the attacker will manage to get at their hands, at least for a few minutes (up to the moment that the victim identifies that their SIM card—which will have been deactivated—does not work and communicates with their provider),



a SIM card with the victim's phone number and could start doing fraud on bank accounts, etc.

The enhanced authentication method proposed in this work is able to effectively thwart such types of attacks, since the attacker needs to have at their hands not only a SIM card with the victim's phone number but also the victim's mobile device.

- **Stolen/lost device:** In case of a device being stolen or lost, then our proposed solution indeed does not provide an extra security layer—i.e., the same risks remain as now, with the currently existing solutions. Of course, for such a threat, other existing safeguards are still applicable, like using biometric authentication to prevent unauthorized access. Further considerations for this issue, as possible future directions, are presented in the Conclusions Section.

#### 6.4. Protocol's Provable Security

As already stated above, our scheme simply adds a new device-based authentication factor to any existing authentication mechanism; these existing mechanisms themselves are not being altered by other means. Therefore, towards proving the security of this new scheme, it suffices to prove the security of the ZKP that utilizes this new authentication factor.

The Feige–Fiat–Shamir protocol that we utilized in our scheme has provable security, as it is shown in [21]. More precisely, we first introduce the following definition (based on [21], being appropriately rephrased here for the sake of simplicity):

**Definition:** An interactive proof of knowledge protocol is unrestricted input zero-knowledge (relative to a trust center) if it satisfies the following:

1. **Works for any input:** The protocol can handle any valid input (statement or data) without requiring pre-approved restrictions or limitations. This means the trust center (a trusted third party) doesn't need to vet or customize the protocol for specific inputs beforehand.
2. **Hides all secret details:** Even when the trust center helps generate initial parameters (like cryptographic keys), the protocol ensures that the verifier does not learn anything about the secret information of the prover beyond the fact that the statement is true. This holds even if the trust center knows the input that is being proven.

Based on the above definition, it is shown in (Theorem 3 [21]) that the Feige–Fiat–Shamir protocol can be unrestricted input zero-knowledge (relative to a trust center).

We next present, for the sake of completeness, a rigorous security proof of the ZKP scheme that we used. First, we show the completeness property—i.e., if the prover knows the secret ( $s_d$ ), the verifier always accepts; this is straightforward from

$$y^2 \equiv (r \cdot s_d^e)^2 \equiv r^2 \cdot s_d^{2e} \equiv x \cdot v_d^e \pmod{N}.$$

Next, regarding soundness, we shall show that if adversary  $\mathcal{A}$  can produce valid responses  $y, y'$  for the same  $x$  and two distinct challenges  $e \neq e'$ , then  $\mathcal{A}$  can efficiently compute the secret key  $s_d$ . In other words, this implies that the ability to answer two distinct challenges for the same witness implies knowledge of the secret key, which is assumed to be hard to compute without factoring  $N$ . Indeed, suppose  $\mathcal{A}$  produces  $y, y'$  such that

$$y^2 \equiv x \cdot v_d^e \pmod{N},$$

$$y'^2 \equiv x \cdot v_d^{e'} \pmod{N}.$$

We next divide the two congruences:

$$\frac{y^2}{y'^2} \equiv \frac{v_d^e}{v_d^{e'}} = v_d^{e-e'} \pmod{N}.$$

Let  $b = e - e' \in \{-1, 1\}$ , and define

$$z = \frac{y}{y'} \pmod{N}.$$

Then,

$$z^2 \equiv v_d^b = s_d^{2b} = \left(s_d^b\right)^2 \pmod{N}.$$

Therefore,

$$z \equiv \pm s_d^b \pmod{N}.$$

Apparently, by computing  $z$  and  $b$ , the adversary  $\mathcal{A}$  can recover the secret  $s_d$ , thus concluding the proof of soundness.

Regarding the zero-knowledge property, we shall prove that the verifier learns nothing about the secret  $(s_d)$  beyond the fact that the prover knows it. To this end, we construct a simulator  $\mathcal{S}$  that, given the public key and a challenge  $e$ , produces a transcript indistinguishable from a real protocol run without knowing the secret.

**Simulator  $\mathcal{S}$ :**

1. Choose random  $y \in \mathbb{Z}_N^*$ .
2. Compute

$$x = y^2 \cdot (v_d^e)^{-1} \pmod{N}.$$

3. Output transcript  $(x, e, y)$ .

In a real protocol,  $x = r^2 \pmod{N}$ , and  $y = r \cdot v_d^e \pmod{N}$ . In the simulation,  $y$  is random, and  $x$  is computed accordingly. Both distributions are uniform over valid triples satisfying

$$y^2 \equiv x \cdot v^e \pmod{N}.$$

Hence, the verifier cannot distinguish simulated transcripts from real ones.

As a last remark, we should recall at this point that our scheme does not necessitate a specific ZKP; other such strong protocols, apart from the Feige–Fiat–Shamir protocol, could also be an option.

## 7. Assessment with Respect to the Personal Data Protection and Privacy Legislation

Both privacy and protection of personal data constitute fundamental human rights in Europe. The most prominent legal instrument is the General Data Protection Regulation, or GDPR [17], which applies to all organizations, regardless of location, if they process the personal data of individuals residing in the European Union. The notion of personal data is defined in the GDPR as a broad term; indeed, even device identifiers should be considered personal data since they may potentially allow the identification of a user when combined with other information.

The GDPR sets the fundamental principles that should be in place for any personal data process. These include purpose limitation, which requires personal data to be collected for specified, explicit, and legitimate purposes and not further processed for other purposes; data minimization, which ensures that the use of the personal data is limited to what is necessary for the purposes for which they are processed; and data security, which requires appropriate technical or organizational measures to protect against unauthorized

or unlawful processing, as well as accidental loss, destruction, or damage. The above are obligations for any data controller as defined within the GDPR; in our case, such a controller is the service provider.

In our enhanced authentication mechanism, the ‘new’ personal information that is being used as an additional authentication factor is the device identifier. However, due to the underlying ZK protocol, this identifier is not being sent or revealed to any other party, including the service provider; this piece of information cannot even be inferred by any party monitoring the communication. Therefore, despite the fact that a device’s unique identifier is necessary for the device’s registration and authentication, it is not being revealed or accessed, nor can it be computed/extracted by other means (and that’s why we considered the honest-but-curious adversarial model for the authentication server). Therefore, the aforementioned data protection principles (i.e., purpose limitation, data minimization, and security) are present in this scheme.

Another fundamental principle for personal data processing, according to the GDPR, is the so-called storage limitation principle, meaning that data should be kept in a form that permits the identification of individuals for no longer than is necessary for the purposes for which the personal data are processed. Again, this principle is fulfilled in our approach; in comparison with what a server already has for authentication purposes (such as, e.g., a user’s login name, etc.), our approach necessitates the storage of a ZKP–public key associated with a valid user’s device, as well as of a timestamp concerning the registration. These pieces of information are necessary and not excessive towards performing (device) authentication, and thus, there is a need to store them in this form while also allowing the server to be accountable as the GDPR requires in art—i.e., to prove, if required, when a specific user’s device has been registered. Moreover, although of course the service provider has the obligation to implement appropriate technical and organizational measures to protect these data, it should be stressed that there are no significant additional risks for the users in case these data are leaked (since any third party getting access to these data still does not have access to the device’s secret key and, thus, cannot masquerade as the user).

It should be pointed out that specific data processes fall under the scope of the legal framework outlined in the ePrivacy Directive (Directive 2002/58/EC) [38], which applies, in those cases, as *lex specialis* to the GDPR. Concerning specifically our authentication scheme, emphasis should be put on art. 5(3) of the e-Privacy Directive, which states that the storing of information, or the gaining of access to information already stored, in the terminal equipment of a subscriber or user’ is only allowed on the basis of consent or necessity for specific purposes set out in this article. In our scheme, the server—due to the ZK protocol—does not gain access to information stored on the user’s device. Therefore, the proposed approach is also in line with this legal provision.

## 8. Comparative Study

Our preliminary experiments illustrate the potential of the proposed approach for efficient implementations; of course, an extra authentication layer introduces additional workload, but it seems that these delays can be acceptable, taking into account the additional levels of security that are provided. In any case, the security parameter  $k$  also affects the performance, so it is up to the service provider to choose a proper  $k$  so as to strike a proper balance between security and performance.

In order to have a comparative study with respect to other ZKPs, we use the relevant information from the European Commission [39], which has very recent performance results regarding the proof generation times of ZKPs. More precisely, this report presents performance results on the following ZKPs: (a) BBS+/CL-Signatures/PS-MS (first introduced in [40] and having been improved later on, with the most recent advancement

in [41]), (b) BBS# [42], (c) Anonymous Credentials from ECDSA [43], (d) Crescent [44], and (e) ZKAttest [45]. Of course, these implementations focus on ZK-based digital signatures, which is different from the use of the Feige–Fiat–Shamir in our framework as a proof of identity scheme (in our case, the proof of device identity is either accepted or rejected, without having messages, judges, or disputes). Moreover, in the aforementioned report, only proof generation times are being considered and not networking conditions or proof verification times. However, it is of interest to see what is considered state-of-the-art in terms of creating ZK-based digital signatures.

The relevant results are given in Table 1.

**Table 1.** Proof generation times of ZKPs (based on [39]).

|      | <b>BBS+/CL-Signatures<br/>/PS-MS</b> | <b>BBS#</b> | <b>Anonymous Credentials<br/>from ECDSA</b> | <b>Crescent</b> | <b>ZKAttest</b> |
|------|--------------------------------------|-------------|---------------------------------------------|-----------------|-----------------|
| Time | ~ms                                  | ~ms         | <2 s                                        | <5 s            | <10 s           |

Our simulations, which additionally take into consideration various network conditions, illustrate that the overall time that is required by our scheme in the case of using a Feige–Fiat–Shamir ZK is in fact less than 2 s in ideal communication conditions and less than 4.5 s for bad networking conditions even for  $k = 10$  iterations (see Figure 8), we get that this time duration is in line with what is considered state-of-the-art. However, once again, we should recall that our scheme is not restricted to any specific ZKP; therefore, other ZKPs could also be examined as possible options.

## 9. Conclusions

This paper presents a framework for implementing an authentication mechanism based on zero-knowledge proofs to enhance existing multi-factor authentication settings based on mobile devices (independently from the underlying platform). Therefore, current risks that are present in existing settings are being alleviated. The important property of the proposed solution is that it does not aim to replace any existing authentication method; instead, it provides an additional—to the already existing—authentication method without sacrificing the user’s privacy while simultaneously being efficient, as illustrated in practice. To this end, we assume that in the case of a Feige–Fiat–Shamir ZKP, a number of about 8–10 iterations of the protocol seems to achieve a nice balance between security and performance (also taking into account that this is one out of many authentication factors that should be simultaneously present); for example, for  $k = 10$  iterations, the probability that an attacker bypasses this new authentication factor becomes less than  $0.98 \times 10^{-3}$ .

Moreover, it is a ‘pluggable’ solution in the sense that the underlying zero-knowledge protocol could be chosen arbitrarily without limitations. Moreover, it is fairly easy to switch the zero-knowledge algorithm to another one. This aspect is also important in the context of the upcoming quantum computing since any post-quantum secure zero-knowledge protocol could also be adopted for this scheme.

In principle, the low complexity and cost of such a scheme, in combination with the security benefits, make it a perfect candidate for a client-server distributed system that requires quick response times and efficiency with regard to energy. Indeed, the inherent properties of the underlying cryptographic zero-knowledge protocol allow us to efficiently strengthen the overall authentication mechanism—since this new scheme provides security against attacks like, e.g., SIM swapping attacks or social engineering attacks—without compromising users’ privacy and personal data. Therefore, recalling our research questions, we may state that zero-knowledge cryptographic protocols can indeed be incorporated

into existing multi-factor authentication (MFA) frameworks, making effective use of the device's identifier so as to provide efficient solutions with tolerable additional workload.

This work opens further research directions; a fully workable end-to-end solution based on multi-factor authentication that makes use of our proposed idea is still needed, which in turn could lead to possible amendments for enhancing, e.g., performance. To this end, ZK-SNARKs could also be considered as a candidate to support this new framework. Other important aspects of the overall architecture also need to be meticulously considered; for example, in case a user's device is stolen or lost, the current solution simply relies on already existing protection techniques (such as locking a device with biometric identification). Therefore, it is essential to further consider solutions to this; a possible direction is to allow the user, through the registration process, to declare somehow her/his specific device with the associated ZKP–public key, in a way that she/he will have the possibility later on, through a web service (like, let's say, a 'panic button service'), to declare that the device has been stolen; since each device is uniquely associated with a public key that is being registered to the service provider, such an implementation seems to be feasible. Finally, other ways to further strengthen the unpredictability of the device's private key could also be considered; for example, such an alternative solution could be one having the registration timestamp in addition to the device identifier as a secret that is being used to generate the device's private key.

**Author Contributions:** T.S. and K.L. contributed to the conceptualization and methodology. The experiments were set up and performed by T.S. Both authors contributed to the writing. All authors have read and agreed to the published version of the manuscript.

**Funding:** This research received no external funding.

**Data Availability Statement:** Data are contained within the article.

**Acknowledgments:** The authors would like to thank the anonymous reviewers for their very constructive comments and suggestions, which helped to greatly improve the manuscript.

**Conflicts of Interest:** The authors declare no conflicts of interest.

## References

- Schneier, B. *Secrets and Lies—Digital Security in a Networked World*; Wiley: Hoboken, NJ, USA, 2004.
- Zaoui, M.; Belfaik, Y.; Sadqi, Y.; Maleh, Y.; Ouazzane, K. A Comprehensive Taxonomy of Social Engineering Attacks and Defense Mechanisms: Toward Effective Mitigation Strategies. *IEEE Access* **2024**, *12*, 72224–72241. [\[CrossRef\]](#)
- Mujeye, S. A Survey on Multi-Factor Authentication Methods for Mobile Devices. In Proceedings of the 2021 4th International Conference on Software Engineering and Information Management, New York, NY, USA, 16–18 January 2021; ICSIM '21; pp. 199–205. [\[CrossRef\]](#)
- Ma, S.; Feng, R.; Li, J.; Liu, Y.; Nepal, S.; Diethelm; Bertino, E.; Deng, R.H.; Ma, Z.; Jha, S. An empirical study of SMS one-time password authentication in Android apps. In Proceedings of the 35th Annual Computer Security Applications Conference, New York, NY, USA, 9–13 December 2019. ACSAC '19; pp. 339–354. [\[CrossRef\]](#)
- ENISA. *Implementation Guidance on NIS 2 Security Measures—Draft for Consultation*; Technical Report; European Union Agency for Cybersecurity: Athens, Greece, 2024.
- Schneier, B. Two-factor authentication: Too little, too late. *Commun. ACM* **2005**, *48*, 136. [\[CrossRef\]](#)
- Lee, K.; Kaiser, B.; Mayer, J.; Narayanan, A. An empirical study of wireless carrier authentication for SIM swaps. In Proceedings of the Sixteenth USENIX Conference on Usable Privacy and Security—SOUPS'20, Boston, MA, USA, 7–11 August 2020.
- Wang, Q.; Wang, D. Understanding Failures in Security Proofs of Multi-Factor Authentication for Mobile Devices. *IEEE Trans. Inf. Forensics Secur.* **2023**, *18*, 597–612. [\[CrossRef\]](#)
- Gehring, E. Choosing passwords: Security and human factors. In *Social Implications of Information and Communication Technology: Proceedings of the IEEE 2002 International Symposium on Technology and Society (ISTAS'02)*, Raleigh, NC, USA, 6–8 June 2002; Proceedings (Cat. No. 02CH37293); IEEE: Piscataway, NJ, USA, 2002; pp. 369–373. [\[CrossRef\]](#)
- Verizon. *Data Breach Investigations Report*; Report; Verizon: New York, NY, USA, 2021.



11. Kato, K.; Klyuev, V. Strong passwords: Practical issues. In Proceedings of the 2013 IEEE 7th International Conference on Intelligent Data Acquisition and Advanced Computing Systems (IDAACS), Berlin, Germany, 12–14 September 2013; Volume 2, pp. 608–613. [\[CrossRef\]](#)
12. Mannuela, I.; Putri, J.; Michael; Anggreainy, M.S. Level of Password Vulnerability. In Proceedings of the 2021 1st International Conference on Computer Science and Artificial Intelligence (ICCSAI), Jakarta, Indonesia, 28 October 2021; Volume 1, pp. 351–354. [\[CrossRef\]](#)
13. Yan, Q.; Han, J.; Li, Y.; Zhou, J.; Deng, R.H. Leakage-resilient password entry: Challenges, design, and evaluation. *Comput. Secur.* **2015**, *48*, 196–211. [\[CrossRef\]](#)
14. Syafitri, W.; Shukur, Z.; Mokhtar, U.A.; Sulaiman, R.; Ibrahim, M.A. Social Engineering Attacks Prevention: A Systematic Literature Review. *IEEE Access* **2022**, *10*, 39325–39343. [\[CrossRef\]](#)
15. Xu, T.; Singh, K.; Rajivan, P. Personalized persuasion: Quantifying susceptibility to information exploitation in spear-phishing attacks. *Appl. Ergon.* **2023**, *108*, 103908. [\[CrossRef\]](#) [\[PubMed\]](#)
16. Gilsean, C.; Shakir, F.; Alomar, N.; Egelman, S. Security and Privacy Failures in Popular 2FA Apps. In Proceedings of the 32nd USENIX Security Symposium, USENIX Security 2023, Anaheim, CA, USA, 9–11 August 2023; Calandrino, J.A., Troncoso, C., Eds.; USENIX Association: Berkeley, CA, USA, 2023; pp. 2079–2096.
17. EU. Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation). *Off. J. Eur. Union—Legis. Ser.* **2016**, *119*, 1–88.
18. Goldwasser, S.; Micali, S.; Rackoff, C. The knowledge complexity of interactive proof-systems. In Proceedings of the Seventeenth Annual ACM Symposium on Theory of Computing—STOC '85, New York, NY, USA, 6–8 May 1985; pp. 291–304. [\[CrossRef\]](#)
19. Limnietis, K. Cryptography as the Means to Protect Fundamental Human Rights. *Cryptography* **2021**, *5*, 34. [\[CrossRef\]](#)
20. Fiat, A.; Shamir, A. How to prove yourself: Practical solutions to identification and signature problems. In Proceedings of the Proceedings on Advances in Cryptology—CRYPTO '86, 11–15 August 1986, pp. 186–194.
21. Feige, U.; Fiat, A.; Shamir, A. Zero-Knowledge Proofs of Identity. *J. Cryptol.* **1988**, *1*, 77–94. [\[CrossRef\]](#)
22. Liu, W.; Wang, X.; Peng, W. Secure Remote Multi-Factor Authentication Scheme Based on Chaotic Map Zero-Knowledge Proof for Crowdsourcing Internet of Things. *IEEE Access* **2020**, *8*, 8754–8767. [\[CrossRef\]](#)
23. Soewito, B.; Marcellinus, Y. IoT security system with modified Zero Knowledge Proof algorithm for authentication. *Egypt. Inform. J.* **2021**, *22*, 269–276. [\[CrossRef\]](#)
24. Chen, Z.; Jiang, Y.; Song, X.; Chen, L. A Survey on Zero-Knowledge Authentication for Internet of Things. *Electronics* **2023**, *12*, 1145. [\[CrossRef\]](#)
25. Hegde, N.; Manvi, S.S. MFZKAP: Multi Factor Zero Knowledge Proof Authentication for Secure Service in Vehicular Cloud Computing. In Proceedings of the 2019 Second International Conference on Advanced Computational and Communication Paradigms (ICACCP), Gangtok, India, 25–28 February 2019; pp. 1–6. [\[CrossRef\]](#)
26. Ben-Sasson, E.; Chiesa, A.; Tromer, E.; Virza, M. Succinct non-interactive zero knowledge for a von Neumann architecture. In Proceedings of the 23rd USENIX Conference on Security Symposium, San Diego, CA, USA, 20–22 August 2014; SEC'14; pp. 781–796.
27. Ben-Sasson, E.; Chiesa, A.; Garman, C.; Green, M.; Miers, I.; Tromer, E.; Virza, M. Zerocash: Decentralized Anonymous Payments from Bitcoin. In Proceedings of the 2014 IEEE Symposium on Security and Privacy, San Jose, CA, USA, 18–21 May 2014; pp. 459–474. [\[CrossRef\]](#)
28. Ben-Sasson, E.; Bentov, I.; Horesh, Y.; Riabzev, M. Scalable, transparent, and post-quantum secure computational integrity. *IACR Cryptol. ePrint Arch.* **2018**, *46*. Available online: <https://eprint.iacr.org/2018/046> (accessed on 25 April 2025).
29. EC. Call for Tenders: Development, Consultancy and Support for an Age Verification Solution. 2024. Available online: <https://digital-strategy.ec.europa.eu/en/funding/call-tenders-development-consultancy-and-support-age-verification-solution> (accessed on 25 April 2025).
30. Gamma, E.; Helm, R.; Johnson, R.; Vlissides, J.; Booch, G. *Design Patterns: Elements of Reusable Object-Oriented Software*; Addison-Wesley Longman Publishing Co., Inc.: St, Boston, MA, USA, 1995.
31. Segkoulis, T. Zkp Service. Github Repository. 2025. Available online: [https://github.com/Jranco/zkp\\_service](https://github.com/Jranco/zkp_service) (accessed on 27 April 2025).
32. Segkoulis, T. Zkp Client. Github Repository. 2025. Available online: <https://github.com/Jranco/zkp-client> (accessed on 27 April 2025).
33. Segkoulis, T. Zkp Client Demo. Github Repository. 2025. Available online: <https://github.com/Jranco/zkp-client-demo> (accessed on 27 April 2025).
34. Dolev, D.; Yao, A. On the security of public key protocols. *IEEE Trans. Inf. Theory* **1983**, *29*, 198–208. [\[CrossRef\]](#)
35. Mayrhofer, R.; Sigg, S. Adversary Models for Mobile Device Authentication. *ACM Comput. Surv.* **2022**, *54*, 198:1–198:35. [\[CrossRef\]](#)



36. Aumann, Y.; Lindell, Y. Security Against Covert Adversaries: Efficient Protocols for Realistic Adversaries. In Proceedings of the Theory of Cryptography, Amsterdam, The Netherlands, 21–24 February 2007; Vadhan, S.P., Ed.; Springer: Berlin/Heidelberg, Germany, 2007; pp. 137–156.
37. Pradeep, A.; Paracha, M.T.; Bhowmick, P.; Davanian, A.; Razaghpanah, A.; Chung, T.; Lindorfer, M.; Vallina-Rodriguez, N.; Levin, D.; Choffnes, D. A comparative analysis of certificate pinning in Android & iOS. In Proceedings of the 22nd ACM Internet Measurement Conference, New York, NY, USA, 25–27 October 2022; IMC '22; pp. 605–618. [CrossRef]
38. EC. Directive 2002/58/EC of the European Parliament and of the Council of 12 July 2002 concerning the processing of personal data and the protection of privacy in the electronic communications sector (Directive on privacy and electronic communications). *Off. J. Eur. Communities* **2002**, 201/37. Available online: <https://eur-lex.europa.eu/eli/dir/2002/58/oj/eng> (accessed on 25 April 2025).
39. EC. Digital Identity for all Europeans—The Official GitHub Organization of the European Digital Identity Project (Annex 2). Report. 2025. Available online: <https://github.com/eu-digital-identity-wallet/av-doc-technical-specification/blob/main/docs/annexes/annex-2/annex-2-high-level-requirements.md> (accessed on 25 April 2025).
40. Boneh, D.; Boyen, X.; Shacham, H. Short Group Signatures. In *Advances in Cryptology—CRYPTO 2004: Proceedings of the 24th Annual International Cryptology Conference, Santa Barbara, CA, USA, 15–19 August 2004*; Proceedings; Franklin, M.K., Ed.; Lecture Notes in Computer Science; Springer: Berlin/Heidelberg, Germany, 2004; Volume 3152, pp. 41–55.
41. Tessaro, S.; Zhu, C. Revisiting BBS Signatures. Cryptology ePrint Archive, Paper 2023/275. 2023. Available online: <https://eprint.iacr.org/2023/275> (accessed on 25 April 2025).
42. Desmoulins, N.; Dumanois, A.; Kane, S.; Traoré, J. Making BBS Anonymous Credentials eIDAS 2.0 Compliant. Cryptology ePrint Archive, Paper 2025/619. 2025. Available online: <https://eprint.iacr.org/2025/619> (accessed on 25 April 2025).
43. Frigo, M.; Shelat, A. Anonymous credentials from ECDSA. Cryptology ePrint Archive, Paper 2024/2010. 2024. Available online: <https://eprint.iacr.org/2024/2010> (accessed on 25 April 2025).
44. Paquin, C.; Policharla, G.V.; Zaverucha, G. Crescent: Stronger Privacy for Existing Credentials. Cryptology ePrint Archive, Paper 2024/2013. 2024. Available online: <https://eprint.iacr.org/2024/2013> (accessed on 25 April 2025).
45. Faz-Hernández, A.; Ladd, W.; Maram, D. ZKAttest: Ring and Group Signatures for Existing ECDSA Keys. Cryptology ePrint Archive, Paper 2021/1183. 2021. Available online: <https://eprint.iacr.org/2021/1183> (accessed on 25 April 2025).

**Disclaimer/Publisher’s Note:** The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.